CLASS HANDOUT · NEXTJS-PRODUCTS-APP

React, Next.js, cookies, and API data

Every topic below is paired with the code from this repository, or with the code to type in class. Blocks marked “in the repo now” match the files. Blocks marked “add in class” are the next step.

Contents

1. React, Vite, and Next.js
2. Client rendering and server rendering
3. How a URL chooses a file
4. Login and route protection as they work today
5. Cookie authentication to add
6. Static product data versus the API fetch
7. Wire the list and the detail page to FakeAPI

1. React, Vite, and Next.js

React is the component. Vite only ships that component to the browser. Next.js maps a file to a URL and can run the component on the server.

PLAIN REACT, THE KIND VITE SERVES

```
<!-- index.html: empty until the script runs -->
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>

// main.jsx
import { createRoot } from "react-dom/client";
createRoot(document.getElementById("root")).render(<App />);
```

IN THE REPO NOW · src/app/page.tsx – this file is the URL /

```
import React from "react";

export default function Home() {
  return <div>Home</div>;
}
```

There is no route table. The file path is the route. `pnpm dev` starts Next.js, not Vite.

	Vite + React	**This app**
Dev server	`npm run dev` runs Vite	`pnpm dev` runs Next.js
First HTML	Empty `#root`	The page component's HTML
URL	You configure a client router	Folder under `src/app`

2. Client rendering and server rendering

Client: the browser decides after JavaScript loads

IN THE REPO NOW · src/components/RouteGuard.tsx

```
"use client";

export default function RouteGuard({ mode, children }) {
  const [isChecking, setIsChecking] = React.useState(true);
  const router = useRouter();

  React.useEffect(() => {
    const isAuth = localStorage.getItem("isAuth") === "true";
    if (isAuth && mode === "public") {
      router.replace("/products");
      return;
    } else if (!isAuth && mode === "private") {
      router.replace("/login");
      return;
    } else {
      setIsChecking(false);
    }
  }, [mode, router]);

  if (isChecking) return <p>Checking authentication...</p>;
  return <>{children}</>;
}
```

`useEffect` does not run on the server. The first HTML is "Checking authentication...". The redirect happens only after the browser runs the bundle. `"use client"` means hooks and clicks run in the browser. Next.js still includes the first HTML in the response. Disable JavaScript on `/login` and the form stays visible, but Sign in does nothing. That attach step is hydration.

Server: the HTML is finished before the browser paints

IN THE REPO NOW · src/app/products/page.tsx

```
export const instant = false;

export default async function ProductsPage({ searchParams }: PageProps<"/">) {
  const { page = 1, limit = 10 } = await searchParams;
  const response = await fetch(
    `${process.env.API_BASE_URL}/products?page=${page}&limit=${limit}`,
  );
  const products: ProductResponse = await response.json();

  return (
    <section>
      {products?.data?.map((product) => (
        <ProductCard key={product.id} product={product} />
      ))}
      <Pagination />
    </section>
  );
}
```

This function runs on the Next.js server. `API_BASE_URL` has no `NEXT_PUBLIC_` prefix, so browser code cannot read it. The browser Network tab shows the document, not this `fetch`.

IN THE REPO NOW · Pagination is the client boundary inside that server page

```
"use client";

export default function Pagination() {
  const [_, setPagination] = useQueryStates({
    page: parseAsInteger.withDefault(1),
    size: parseAsInteger.withDefault(5),
  });

  return [1, 2, 3, 4].map((n) => (
    <Button key={n} onClick={() => setPagination({ page: n })}>
      {n}
    </Button>
  ));
}
```

The server renders the buttons. `onClick` arrives with the JavaScript. The root layout stays a Server Component and passes pages through a client provider:

IN THE REPO NOW · src/app/layout.tsx

```
export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html lang="en">
      <body>
        <NuqsAdapter>
          <CartStoreProvider>{children}</CartStoreProvider>
        </NuqsAdapter>
      </body>
    </html>
  );
}
```

3. How a URL chooses a file

No config file maps `/login`. Next.js drops a folder written in parentheses and treats `page.tsx` as the page.

```
src/app/(auth)/login/page.tsx
        ------  -----  --------
        omitted  URL    this file is the page
```

IN THE REPO NOW · src/app/(auth)/login/page.tsx

```tsx
import LoginForm from "@/features/auth/components/LoginForm";

export const instant = true;

export default function LoginPage() {
  return <LoginForm />;
}
```

IN THE REPO NOW · src/app/(auth)/layout.tsx – wraps /login, does not add /auth

```tsx
export default function AuthLayout({ children }: LayoutProps<"/">) {
  return (
    <RouteGuard mode="public">
      <main>
        <section className="bg-red-500">...</section>
        <section>{children}</section>
      </main>
    </RouteGuard>
  );
}
```

Request order for `/login`: `src/app/layout.tsx`, then `(auth)/layout.tsx`, then `login/page.tsx` inside `{children}`.

File	URL
`src/app/page.tsx`	`/`
`src/app/(auth)/login/page.tsx`	`/login`
`src/app/products/page.tsx`	`/products`
`src/app/products/[id]/page.tsx`	`/products/1` sets `id` to `"1"`
`src/app/cart/page.tsx`	`/cart`

`layout.tsx` and `error.tsx` do not create URLs. `router.replace("/login")` only navigates. `LoginForm.tsx` and `login.action.tsx` are imports. Moving them does not change the URL. Renaming the `login` folder to `sign-in` would make the URL `/sign-in`. Removing the parentheses would make it `/auth/login`.

IN THE REPO NOW · src/app/products/[id]/page.tsx – dynamic segment

```tsx
export default async function ProductDetailPage({
  params,
}: PageProps<"/products/[id]">) {
  return (
    <Suspense>
      <ProductDetail params={params} />
    </Suspense>
  );
}
```

4. Login and route protection as they work today

IN THE REPO NOW · src/features/auth/schemas/login.schema.ts

```
export const loginSchema = z.object({
  email: z.email("Enter a valid email"),
  password: z.string().min(1, "Password is required"),
});
```

IN THE REPO NOW · the submit handler in LoginForm.tsx

```
"use client";

const onSubmit = (values: LoginFormValues) => {
  console.log(values);
  if (
    values.email === "sreekanth@gmail.com" &&
    values.password === "123123123"
  ) {
    localStorage.setItem("isAuth", "true");
    router.replace("/products");
  }
};
```

Demo account on the form: `sreekanth@gmail.com` / `123123123`. A wrong password only hits `console.log`. The server never sees the password.

IN THE REPO NOW · src/app/(auth)/layout.tsx passes mode="public"

```
<RouteGuard mode="public">{children}</RouteGuard>
```

Public mode: if `isAuth` is `"true"`, go to `/products`. Otherwise show the form.

IN THE REPO NOW · src/app/products/layout.tsx

```
export default function ProductsLayout({ children }: LayoutProps<"/">) {
  return (
    <RouteGuard mode="public">
      <div>
        <ProductsHeader />
        <main>{children}</main>
      </div>
    </RouteGuard>
  );
}
```

> This layout currently passes `mode="public"`. Logged-out students can open `/products`. Private protection is the other branch of the guard. Change the prop when you want the redirect:

CHANGE THIS ONE PROP TO LOCK /PRODUCTS

```
<RouteGuard mode="private">
```

Private mode runs `router.replace("/login")` when `localStorage` is not `"true"`. `/` and `/cart` have no `RouteGuard`.

The cookie action is already written and the form does not call it. Its account is different: `s@gmail.com` / `123`.

IN THE REPO NOW · src/features/auth/action/login.action.tsx

```
"use server";

export default async function loginAction(values: unknown) {
  const parsed = loginSchema.safeParse(values);
  if (!parsed.success) {
    return { success: false, error: "Please enter valid email and password." };
  }

  const { email, password } = parsed.data;
  const isValidUser = email === "s@gmail.com" && password === "123";
  if (!isValidUser) {
    return { success: false, error: "Invalid email or password." };
  }

  const cookiestore = await cookies();
  cookiestore.set({
    name: "ecom-auth",
    value: "123",
    httpOnly: true,
    sameSite: "lax",
    path: "/",
    maxAge: 60 * 60 * 24,
  });
  return { success: true };
}
```

5. Cookie authentication to add

`httpOnly: true` means page JavaScript cannot read the cookie. `document.cookie` will not list `ecom-auth`. Leave `secure` off on `http://localhost`.

ADD IN CLASS · replace the if inside LoginForm onSubmit. Keep react-hook-form. Do not use form action={loginAction}, because that passes FormData and the Zod parse fails.

```
const [serverError, setServerError] = useState("");

const onSubmit = async (values: LoginFormValues) => {
  setServerError("");
  const result = await loginAction(values);
  if (!result.success) {
    setServerError(result.error);
    return;
  }
  router.replace("/products");
};
```

ADD IN CLASS · src/features/auth/session.ts

```
import "server-only";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function getSession() {
  const token = (await cookies()).get("ecom-auth")?.value;
  if (token !== "123") return null;
  return { userId: "student" };
}

export async function requireUser() {
  const session = await getSession();
  if (!session) redirect("/login");
  return session;
}
```

ADD IN CLASS · top of src/app/products/layout.tsx. Remove RouteGuard. Add export const instant = false if prerender complains.

```
export const instant = false;

export default async function ProductsLayout({ children }: LayoutProps<"/">) {
  await requireUser();
  return (
    <div className="flex grow flex-col bg-neutral-50 text-neutral-900">
      <ProductsHeader />
      <main>{children}</main>
    </div>
  );
}
```

ADD IN CLASS · src/app/(auth)/layout.tsx

```
export default async function AuthLayout({ children }: LayoutProps<"/">) {
  const session = await getSession();
  if (session) redirect("/products");
  return <main>{children}</main>;
}
```

ADD IN CLASS · src/proxy.ts – early redirect only. This file does not create /login.

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  const isLoggedIn = request.cookies.get("ecom-auth")?.value === "123";
  const { pathname } = request.nextUrl;

  if (pathname.startsWith("/products") && !isLoggedIn) {
    return NextResponse.redirect(new URL("/login", request.url));
  }
  if (pathname === "/login" && isLoggedIn) {
    return NextResponse.redirect(new URL("/products", request.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: ["/products/:path*", "/login"],
};
```

ADD IN CLASS · logout Server Action, called from ProductsHeader

```
"use server";

import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function logoutAction() {
  (await cookies()).delete("ecom-auth");
  redirect("/login");
}
```

> The value `"123"` is a shared password in source. The following lesson signs `{ userId }` with `jose` and `SESSION_SECRET` instead of storing `"123"`.

6. Static product data versus the API fetch

IN THE REPO NOW · src/constants/data.ts – 20 items, field name is name

```
export type Product = {
  id: number;
  name: string;
  description: string;
  image: string;
  price: number;
};

export const products: Product[] = [
  {
    id: 1,
    name: "iPhone 16 Pro",
    description: "Premium smartphone with an advanced camera and A18 Pro chip.",
    image: "https://images.unsplash.com/photo-1592899677977-9c10ca588bbd?w=600",
    price: 999,
  },
];
```

IN THE REPO NOW · ProductDetail.tsx reads that array. No network.

```
const { id } = await params;
const product = products.find((item) => item.id === Number(id));
```

IN THE REPO NOW · src/app/products/[id]/page.tsx also searches the array for the title

```
async function getProduct(id: string) {
  "use cache";
  return products.find((p) => p.id === Number(id));
}
```

IN THE REPO NOW · .env.local

```
API_BASE_URL=https://fakeapi.net/docs/fake-e-commerce-api
```

> That value is the documentation page. The API base URL on that page is `https://fakeapi.net`. With the docs URL, the list page requests `https://fakeapi.net/docs/fake-e-commerce-api/products?page=1&limit=10`, which is not the products endpoint.

CORRECT .ENV.LOCAL

```
API_BASE_URL=https://fakeapi.net
```

Restart `pnpm dev` after editing it. The list page then requests:

```
GET https://fakeapi.net/products?page=1&limit=10
```

FakeAPI returns this shape. It matches `Product.title` on the list page and `ProductCard`:

```
{
  "data": [
    {
      "id": 1,
      "title": "Smartphone X",
      "price": 799.99,
      "description": "Latest smartphone with advanced features",
      "image": "https://fakeapi.net/images/smartphone.jpg"
    }
  ],
  "pagination": { "page": 1, "limit": 10, "total": 20 }
}
```

IN THE REPO NOW · ProductCard reads title, the static file uses name

```
<h2 className="font-medium">{product.title}</h2>
<p>${product.price}</p>
```

Pagination writes `size`. The server reads `limit`. Clicking page 2 produces `?page=2&size=5`, and the fetch still uses `limit=10`.

IN THE REPO NOW, UNUSED BY THE ROUTE · Products.tsx slices the static array in the browser

```
"use client";
const filteredProducts = products.slice(startIndex, endIndex);
```

7. Wire the list and the detail page to FakeAPI

ADD IN CLASS · src/features/products/api.ts

```
const base = process.env.API_BASE_URL;

export type Product = {
  id: number;
  title: string;
  description: string;
  image: string;
  price: number;
};

type ListResponse = {
  data: Product[];
  pagination: { page: number; limit: number; total: number };
};

export async function getProducts(page: number, limit: number) {
  const url = new URL("/products", base);
  url.searchParams.set("page", String(page));
  url.searchParams.set("limit", String(limit));

  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`Products request failed (${response.status})`);
  }
  return (await response.json()) as ListResponse;
}

export async function getProduct(id: string) {
  const response = await fetch(new URL(`/products/${id}`, base));
  if (response.status === 404) return null;
  if (!response.ok) {
    throw new Error(`Product request failed (${response.status})`);
  }
  return (await response.json()) as Product;
}
```

ADD IN CLASS · src/app/products/page.tsx

```
export default async function ProductsPage({ searchParams }: PageProps<"/">) {
  const params = await searchParams;
  const page = Number(params.page ?? 1);
  const limit = Number(params.limit ?? 10);
  const { data, pagination } = await getProducts(page, limit);

  return (
    <section>
      <h1 className="mb-6 text-2xl font-semibold">Products</h1>
      <div className="grid grid-cols-1 gap-5 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4">
        {data.map((product) => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
      <Pagination page={pagination.page} limit={pagination.limit} total={pagination.total} />
    </section>
  );
}
```

ADD IN CLASS · Pagination.tsx — same query name the server reads

```
"use client";

export default function Pagination({
  page,
  limit,
  total,
}: {
  page: number;
  limit: number;
  total: number;
}) {
  const [, setPagination] = useQueryStates({
    page: parseAsInteger.withDefault(1),
    limit: parseAsInteger.withDefault(10),
  });
  const pageCount = Math.ceil(total / limit);

  return (
    <div>
      {Array.from({ length: pageCount }, (_, index) => (
        <Button key={index} onClick={() => setPagination({ page: index + 1 })}>
          {index + 1}
        </Button>
      ))}
    </div>
  );
}
```

ADD IN CLASS · ProductDetail.tsx – replace the static find

```
const { id } = await params;
const product = await getProduct(id);

if (!product) {
  return (
    <section>
      <p>Product not found.</p>
      <Link href="/products">Back to products</Link>
    </section>
  );
}
```

Render `product.title`, because FakeAPI uses `title`. The static detail page renders `product.name`. Use one field after the switch. Call the same `getProduct` inside `generateMetadata` and delete the `"use cache"` search of `data.ts`.

IN THE REPO NOW · src/app/products/error.tsx – shown when getProducts throws

```
"use client";

export default function Error({
  error,
  retry,
}: {
  error: unknown;
  retry: () => void;
}) {
  React.useEffect(() => {
    console.error(error);
  }, [error]);

  return (
    <div>
      <h2>Something went wrong!</h2>
      <button onClick={() => retry()}>Try again</button>
    </div>
  );
}
```

A missing id should return the “Product not found” branch, not throw. An empty `data` array should render “No products”.

After the cart and `CartControls` use the API `Product` type (`title`, not `name`), delete `src/constants/data.ts`. Deleting it earlier breaks the cart import.